Oracle Data Governance — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Oracle Data Governance: Microsoft Purview 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 5

Outcome . . . 5

1. Solution Overview — Oracle Data Governance: Microsoft Purview 5

What this solution is . . . 5

Why this matters . . . 5

How Purview reaches Oracle . . . 5

The Data Map, collections, and roles . . . 6

Credentials and network — safe by design . . . 6

Licensing and the evolving portal . . . 6

What this is not . . . 7

The deployment model . . . 7

2. Requirements Guide — Oracle Data Governance: Microsoft Purview 7

2.1 A Microsoft Purview account (Data Map) provisioned on an Azure subscription . . . 7

2.2 Oracle connector availability + supported version confirmed for your tenant . . . 8

2.3 Least-privilege data-governance roles on the Data Map collection . . . 8

2.4 An inventory of the Oracle databases/schemas in scope, with owners . . . 8

2.5 A least-privilege, read-only Oracle database login for scanning . . . 8

2.6 The Oracle credential stored in Azure Key Vault — never in code . . . 9

2.7 A self-hosted integration runtime (SHIR) + network path + JDBC driver . . . 9

2.8 A classification approach (system + any custom classifications) . . . 9

2.9 A pilot scope and change control . . . 9

Checklist . . . 10

Global Requirements — Microsoft Purview Solutions 10

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11

2. Purview roles (least-privilege) and portal access . . . 11

3. Security & Compliance PowerShell + Graph toolchain . . . 11

4. Data sources & scope in scope . . . 11

5. Compliance, records, privacy & legal stakeholders . . . 12

6. A pilot scope and change control . . . 12

3. Implementation Guide — Oracle Data Governance: Microsoft Purview 12

Step 1 — Assess current state (read-only) . . . 13

Step 2 — Design the collection and role model . . . 13

Step 3 — Create the least-privilege Oracle login (read-only) . . . 13
Step 4 — Establish the credential path (Key Vault, safe by design) . . . 13
Step 5 — Establish the SHIR, network path, and driver . . . 13
Step 6 — Record the pilot scope . . . 14
Step 7 — Register the pilot Oracle source into the Data Map . . . 14
Step 8 — Configure and pilot the scan (bounded scope, pilot-first) . . . 14
Step 9 — Review scanned assets and classifications . . . 14
Step 10 — Assign ownership on the Oracle assets . . . 15
Step 11 — Validate against the pilot gate . . . 15
Step 12 — Expand across the Oracle estate (enforce) . . . 15
5. Testing Guide — Oracle Data Governance: Microsoft Purview 15
Test 1 — Account and roles are ready . . . 16
Test 2 — The Oracle connector is available . . . 16
Test 3 — A SHIR is registered and running . . . 16
Test 4 — The credential path works (Key Vault + Test connection) . . . 16
Test 5 — The scan succeeds and returns assets . . . 16
Test 6 — Oracle assets are classified . . . 16
Test 7 — Scanned Oracle assets have owners . . . 16
Test 8 — Consolidated readiness gate . . . 17
Pass/fail gate . . . 17
6. Troubleshooting Guide — Oracle Data Governance (Microsoft Purview) 17
A. Account, collections & roles . . . 17
B. Connector availability . . . 17
C. SHIR / network / driver . . . 18
D. Credential path . . . 18
E. Scan results . . . 18
F. Classification & ownership . . . 19
Escalation checklist . . . 19
7. Best Practices — Oracle Data Governance: Microsoft Purview 19
8. Deliverables Summary — Oracle Data Governance: Microsoft Purview 21
Documentation . . . 21
Named artifacts . . . 21
PowerShell scripts . . . 21
What the buyer receives . . . 22
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Oracle Data Governance: Microsoft Purview

Most data-governance efforts stop at Microsoft 365, and the Oracle estate — customer masters, ERP and line-of-business schemas, data-warehouse staging — sits outside the map, unclassified and ungoverned. You cannot protect, retain, or grant access to data you cannot see. This solution closes that blind spot: it **registers, scans, and classifies your Oracle databases into the Microsoft Purview Data Map**, so Oracle data becomes a mapped, classified, owned part of the same governed estate as the rest of your data.

This is a complete, do-it-today package written for an engineer bringing Oracle into Purview for the first time. It delivers the Oracle connector workflow end-to-end — a least-privilege read-only database login, a **Key Vault** credential referenced by name, a **self-hosted integration runtime** with the Oracle JDBC driver, a bounded pilot scan, classification review, and ownership — with every artifact named once and reused everywhere, and every scan proven on a bounded pilot before the Oracle estate is touched. Because the Purview connectors are evolving and consumption-billed, the guide cites the method and flags every capability "verify your licensing/tenant" rather than asserting a connector, a supported version, or a price as permanent.

What you get

- A plain-language Solution Overview of the Oracle source — how Purview reaches Oracle (host/service on

the listener port through a SHIR), how it authenticates (Basic auth via a Key Vault secret referenced by name), how it scans and classifies schemas/tables/columns, and how it complements Microsoft 365 Purview.

- A Requirements Guide with verify / implement-if-missing for every prerequisite: the Purview account,

connector and version support, least-privilege collection roles, the Oracle inventory with owners, a least-privilege read login, the Key Vault credential, the SHIR + network path + JDBC driver, a classification approach, and the pilot scope.

- An Implementation Guide of precise, ordered steps: design the collection/role model, create the read

login, establish the Key Vault credential and SHIR/network/driver safely, register the Oracle source, **pilot-scan** a bounded set of schemas, review classifications, assign ownership, validate against the gate, then expand with scheduled scans.

- Five PowerShell tools for readiness/role discovery, pilot-scope capture, Oracle source registration

and scan configuration (`-WhatIf` by default), a scan-and-classification coverage report, and a consolidated readiness-gate check — authentic Purview data-plane REST automation with **no credentials in code** (the Oracle secret referenced by Key Vault name only).

- Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

Oracle data governed from Purview: databases and schemas registered into the Data Map, scanned and classified, and owned — validated on a pilot before you scale.

The deployment model

Assess → Design → Configure → **Pilot (bounded scan)** → Validate → Expand — with a pilot gate before any Oracle-wide scanning.

Outcome

The Oracle blind spot closed — databases and schemas discoverable, classified, owned, and governed in the same Purview Data Map as the rest of the estate — with regulatory, records, and access determinations left to the owners who make them.

1. Solution Overview — Oracle Data Governance: Microsoft Purview

What this solution is

Microsoft Purview has two related but distinct sides. The **compliance** side (labels, DLP, retention, insider risk) governs Microsoft 365 content. This solution implements a focused slice of the **data governance** side: **registering, scanning, and classifying your Oracle databases into the Purview Data Map**, so Oracle data stops being a governance blind spot and becomes a mapped, classified, cataloged part of your estate. It does not re-implement the whole Data Map / Unified Catalog build; it delivers the **Oracle-source connector workflow** end-to-end: register the source, prove the credential and network path, run a bounded pilot scan, review classifications, then expand.

> ***Boundary.*** *This solution provides the technical method to *register, scan, classify, and catalog* Oracle sources. It does **not** make the business or regulatory determinations — which data is "confidential", what the retention or records schedule is, or who may access it — those belong to the organization's data owners, stewards, compliance, privacy, and legal functions. The solution gives them governed, discoverable Oracle data to make those decisions against.*

Why this matters

Many organizations govern Microsoft 365 well and their Oracle estate not at all. A customer master lives in an Oracle schema, an ERP or a line-of-business application runs on Oracle, a data-warehouse staging schema holds regulated extracts — and none of it appears in the data map. Nobody can answer "is there PII in our Oracle databases?", "which schemas hold regulated data?", or "who owns this database?" — because Oracle data was never registered, scanned, or classified. Ungoverned data is un-securable data: you cannot protect, retain, or grant access to something you cannot see. This solution brings Oracle into the same Purview Data Map that already governs the rest of the estate.

How Purview reaches Oracle

Oracle databases are commonly on-premises or in a private network, so this connector is built around a **self-hosted integration runtime (SHIR)** and a **credential referenced from Azure Key Vault by name** —

never a secret in a script. Availability and the exact configuration of the Oracle connector are **evolving** in Microsoft Purview; every capability below is flagged "verify your licensing/tenant".

1. **The source** — Purview registers an **Oracle** data source in the Data Map by its **host** and

service name (or SID) and default listener **port** `1521` (verify your instance's port). One Purview source represents one Oracle server/service; a scan enumerates its **schemas** → **tables** → **columns**.

1. **Authentication** — Purview connects with **Basic authentication**: a least-privilege Oracle

database login whose password is stored in an **Azure Key Vault** linked to the Purview account and referenced **by name**. **No credential is ever placed in a script or document.** (Verify the current supported authentication methods for the Oracle connector in your tenant.)

1. **Network + driver** — because Oracle typically sits behind a firewall, scanning runs through a

self-hosted integration runtime (SHIR) installed on a machine with network line of sight to the Oracle listener. The SHIR machine historically requires the **Oracle JDBC driver** to be present for the scanner (place the `ojdbc` driver where Microsoft's current docs specify) — **verify whether your SHIR version bundles it or requires a manual driver install.** A managed VNet runtime may be an option for cloud-reachable instances — verify.

1. **Classification** — a scan applies **system classifications** (credit card, national ID, and the

large built-in set) plus any **custom classification rule** you define in the scan rule set. Purview can also surface **lineage** from Oracle where supported — verify.

The Data Map, collections, and roles

Oracle sources register into the **Data Map** under a **collection** hierarchy that also carries access control. Each collection carries its own role assignments: **Collection admins** (manage the collection and its roles), **Data source admins** (register sources and run scans), **Data curators** (edit catalog metadata and classifications), and **Data readers** (browse the catalog). Least-privilege role assignment is a first-class step, not an afterthought — the Implementation Guide names each role at each stage.

Credentials and network — safe by design

Scanning reaches into a live Oracle system, so it needs a credential and network line of sight — never a secret in a script. This solution follows the safe-by-design pattern Microsoft documents:

• The Oracle login's secret is stored in an **Azure Key Vault** linked to the Purview account and

referenced **by name** — no password in code, in a scan definition, or in a document.

• The database login is **least-privilege and read-only**: `CREATE SESSION` plus `SELECT` on the

schemas/objects in scope and the data-dictionary views the scanner reads (verify the exact grant list in Microsoft's current Oracle-connector documentation). It is not a DBA account.

• Scans reach Oracle through a **self-hosted integration runtime (SHIR)** with line of sight to the

listener; scans are always **piloted on a bounded scope first** (one database/a few schemas).

Licensing and the evolving portal

Microsoft Purview Data Governance (the Data Map and its connectors) is billed through a **consumption / pay-as-you-go** model tied to an **Azure subscription**, and the capability set is evolving — the **Oracle** connector, its supported Oracle versions, the driver requirement, the supported authentication methods, and the exact scan configuration may be in preview or may have changed name, path, or availability. **Verify current pricing, availability, supported versions, and portal paths for your tenant** in Microsoft's documentation and the Purview portal before you plan. This solution cites the method; it never asserts a price, a version, or a portal path as permanent.

What this is not

This is the *Oracle data-source governance* solution. It is **not** the full Data Map / Unified Catalog build, and it is **not** the Microsoft 365 compliance configuration (sensitivity/retention labels, DLP, insider risk) — those are separate Purview solutions in this category. It complements them by pulling Oracle into the same governed map.

The deployment model

> *Assess Oracle readiness (account, roles, connector availability, credential, SHIR/network, driver), design the collection and role model and the credential path, configure the Oracle source registration, **pilot-scan a bounded scope** (one database / a few schemas), validate the results against the gate, then expand across the Oracle estate — moving scans from one-time to scheduled only after the pilot passes. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.***

2. Requirements Guide — Oracle Data Governance: Microsoft Purview

Verify every requirement before you register an Oracle source or run a scan. Each entry gives Description, Why, How to verify, and How to implement if missing. Category-wide prerequisites (tenant, roles, toolchain, scope, stakeholders, pilot) are in `global-requirements-purview.md` — this guide adds the prerequisites specific to **governing Oracle databases in the Purview Data Map**.

> ***Licensing/availability caveat.** Microsoft Purview Data Governance is billed by a consumption / pay-as-you-go model on an Azure subscription and is evolving; the **Oracle** connector, its supported Oracle versions, the JDBC-driver requirement, the supported authentication methods, and the exact scan configuration may be in preview or may have changed. Where this guide names a capability, a version, or a portal path, **verify its current availability for your tenant** in Microsoft's documentation and the Purview portal before relying on it.*

2.1 A Microsoft Purview account (Data Map) provisioned on an Azure subscription

Description: A Microsoft Purview account — the Data Map resource — provisioned in an **Azure subscription** (distinct from the M365 compliance portal). The account exposes the collection hierarchy, scanning, and the connectors Oracle sources register into. Why: The Data Map and scans live in this account; without it there is nothing to register an Oracle source into. How to verify: The account is visible in the Azure portal and its data-plane endpoint (`https://<account>.purview.azure.com`) resolves; run `DGORACLE-Discovery.ps1` to confirm reachability. How to implement if missing: Provision a

Microsoft Purview account under change control on the target subscription; confirm the consumption billing model and region with your Azure owner.

2.2 Oracle connector availability + supported version confirmed for your tenant

Description: Confirmation that the **Oracle** source connector is available in your Purview account, and that your Oracle database **version** is within the connector's currently supported range (verify — historically a broad range up to 19c, but confirm current support). Why: Connector availability and the supported version range are evolving/preview; you cannot register a source type or scan a version the connector doesn't currently support in your tenant. How to verify: In the Purview portal, **Data Map** → **Sources** → **Register**, confirm **Oracle** appears as a registerable source type; check Microsoft's current Oracle-connector documentation for the supported versions and capabilities (scan, classification, lineage). How to implement if missing: If the connector isn't offered, verify licensing/region and Microsoft's current availability; if your Oracle version is unsupported, plan accordingly and scope the pilot to a supported instance.

2.3 Least-privilege data-governance roles on the Data Map collection

Description: Role assignments at the right scope — **Collection admin**, **Data source admin**, **Data curator**, **Data reader** — on the collection the Oracle source will register into. Azure **Owner/Contributor** on the account only where account-level changes are required. Why: Purview uses fine-grained, collection-scoped roles; registering a source and running a scan requires **Data source admin** on that collection, and least-privilege is a governance requirement in its own right. How to verify: In the Purview portal, open the collection's **Role assignments**; run `DGORACLE-Discovery.ps1` (reports collections and role coverage, read-only). How to implement if missing: Assign the least-privilege role at the narrowest collection scope under change control; reserve account-level Collection admin for a small named group.

2.4 An inventory of the Oracle databases/schemas in scope, with owners

Description: A written list of the **Oracle sources** to govern — the **host/service name (or SID)**, **listener port**, **version**, and the **schemas** in scope on each — each with a named **data owner** and **data steward**. Why: You register, scan, and classify per source; classification and ownership are meaningless without a known owner per database/schema, and the host/service/port drive the connection and network path the scan needs. How to verify: For each in-scope database you can name the host/service, port, version, in-scope schemas, and its owner; run `DGORACLE-Set-Scope.ps1` to record the pilot source set. How to implement if missing: Inventory the Oracle estate with the DBAs, and assign a data owner and steward to every database/schema before registering it.

2.5 A least-privilege, read-only Oracle database login for scanning

Description: An Oracle database login the scanner authenticates as, granted **least privilege and read-only**: `CREATE SESSION` plus `SELECT` on the in-scope schemas/objects and on the data-dictionary views the scanner reads (verify the exact grant list — for example the `ALL_*` catalog views — in Microsoft's current Oracle-connector documentation). It is **not** a DBA/SYS account. Why: The scan authenticates with Basic authentication and needs read access to enumerate schemas, tables, columns, and sample data for classification — but no more. A least-privilege read login is the least the scan needs and the least the source owner should grant. How to verify: The login exists with `CREATE SESSION` and

the required `SELECT` grants; the Oracle source's **Test connection** in Purview succeeds using it. How to implement if missing: With the DBA, create a dedicated read-only login (for example `PURVIEW_SCAN`), grant `CREATE SESSION` and the required `SELECT` privileges, and confirm it can query the in-scope objects and catalog views.

2.6 The Oracle credential stored in Azure Key Vault — never in code

Description: The scanning login's **password stored in an Azure Key Vault** that is **linked to the Purview account**, exposed as a Purview credential (Basic authentication) referenced **by name**. **No credential is ever placed in a script or document.** Why: Oracle scans authenticate with Basic auth; storing the secret in Key Vault and referencing it by name is the mandatory secure pattern and is how the provided scripts stay secret-free. How to verify: A Key Vault is linked to the Purview account and holds the Oracle secret; the Oracle source's **Test connection** succeeds using the named credential. How to implement if missing: Link a Key Vault to the account, store the login's password as a secret, create a Purview credential (Basic auth) that references the Key Vault secret by name, and verify **Test connection**.

2.7 A self-hosted integration runtime (SHIR) + network path + JDBC driver

Description: A working scan path to the Oracle listener: a **self-hosted integration runtime (SHIR)** installed on a machine with **network line of sight** to the Oracle host on the listener port (default `1521`), the SHIR registered to the Purview account and showing **Running/Available**, and the **Oracle JDBC driver** present on the SHIR machine where Microsoft's current docs specify (**verify whether your SHIR version bundles the driver or requires a manual install**). A managed VNet runtime may be an option for cloud-reachable instances — verify. Why: A scan cannot classify what it cannot reach; Oracle behind a firewall requires the SHIR, and the scanner needs the JDBC driver to talk to Oracle. Missing runtime, network, or driver is the most common cause of a failed **Test connection**. How to verify: The SHIR shows **Running/Available** in the portal, the firewall permits the SHIR host to reach the listener port, the JDBC driver is present, and **Test connection** on the source succeeds; `DGORACLE-Discovery.ps1` reports registered runtimes. How to implement if missing: Install and register a SHIR with line of sight to the Oracle listener, open the firewall to the listener port from the SHIR host only, install the Oracle JDBC driver per Microsoft's current docs, then re-test.

2.8 A classification approach (system + any custom classifications)

Description: A decision on how Oracle assets will be classified: rely on the **built-in system classifications** (credit card, national ID, and the large built-in set) and define any **custom classification rules** the organization needs (for example an internal customer-ID or account-number pattern) via the scan rule set. Why: Classification is what turns the raw Oracle inventory into a sensitivity-aware map that drives ownership, access, and downstream protection. How to verify: System classifications appear on a test scan result; any custom classification rule is defined and matches sample data. How to implement if missing: Start with system classifications for the pilot; define custom rules only where a real internal pattern isn't covered, and validate each against sample data before estate-wide use.

2.9 A pilot scope and change control

Description: A **bounded pilot** — one collection and one Oracle source scoped to a **few schemas** — to prove registration, credential, network/driver, scanning, and classification before Oracle-wide rollout, plus a change-control window (scans consume SHIR and database resources and read live data). Why: Every

solution in this category follows Assess → Design → Configure → **Pilot** → Validate → Expand; scanning is resource- and access-sensitive and is always proven on a bounded scope first. How to verify: A named pilot collection, its Oracle source and in-scope schemas, and a change window exist; run `DGORACLE-Set-Scope.ps1` to materialize the pilot scope. How to implement if missing: Choose one well-understood, high-value database and a few schemas as the pilot; agree the scan window and expected load with the DBA and source owner.

Checklist

- Purview account (Data Map) provisioned on an Azure subscription — `DGORACLE-Discovery.ps1`
- Oracle connector availability + supported version confirmed for the tenant
- Least-privilege collection roles assigned (Data source admin to register/scan) — `DGORACLE-Discovery.ps1`
- Oracle databases/schemas inventoried with owners, host/service, port, version — `DGORACLE-Set-Scope.ps1`
- Least-privilege read-only Oracle login created (CREATE SESSION + required SELECT grants)
- Oracle credential stored in Azure Key Vault, referenced by name (no secrets in code)
- SHIR installed/registered (Running), network path to listener open, JDBC driver present — Test connection passes
- Classification approach agreed (system + any custom)
- Pilot scope (one database, a few schemas) and scan change window agreed — `DGORACLE-Set-Scope.ps1`

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Oracle Data Governance: Microsoft Purview

Follow these steps in order. This guide assesses Oracle readiness, designs the collection and role model, establishes the credential and SHIR/network path safely, registers and **pilot-scans** a bounded Oracle source (one database, a few schemas), reviews classifications, then expands across the Oracle estate. Artifact names are fixed — create them with the exact names shown and reuse them everywhere. **Every scan is piloted on a bounded scope before the Oracle estate is scanned.**

> *Model: prove the Oracle connector, credential path, network/driver, and classification on a pilot scope, then expand. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.** Verify current portal paths, connector availability, supported Oracle versions, the driver requirement,*

and the supported authentication methods for your tenant as you go — the connector is evolving.

Step 1 — Assess current state (read-only)

1. Run `DGORACLE-Discovery.ps1` to capture readiness signals: the Purview account endpoint is reachable, which **collections** exist and their **role coverage**, which **integration runtimes** are registered (you need a SHIR), and how many Oracle (and other) sources are already registered. No changes are made.
1. In the Purview portal (**Data Map** → **Sources** → **Register**) confirm **Oracle** is offered and note the **supported versions and capabilities** (scan, classification, lineage) for your tenant.
1. Record gaps against the Requirements checklist. Do **not** proceed past a missing account, a missing connector, missing least-privilege roles, a missing credential, or a missing SHIR/network/driver — remediate those first.

Step 2 — Design the collection and role model

1. Decide the **collection** the Oracle source registers into. For the pilot, create one collection named `DGORACLE-Collection-Pilot` under the root; plan the broader hierarchy (for example an `Oracle` branch by database or by sensitivity) on paper for the expansion phase.
1. Assign least-privilege roles at the narrowest scope: **Data source admin** on `DGORACLE-Collection-Pilot` to whoever will register/scan the Oracle source, **Data curator** to the catalog editors, **Data reader** to browsers, and **Collection admin** to a small named group. Reserve account-level admin for a minimal named group.

Step 3 — Create the least-privilege Oracle login (read-only)

1. With the DBA, create a dedicated read-only scanning login (for example `PURVIEW_SCAN`).
2. Grant it `CREATE SESSION` and `SELECT` on the in-scope schemas/objects and the data-dictionary views the scanner reads (verify the exact grant list in Microsoft's current Oracle-connector documentation). **Do not** grant DBA/SYS. Confirm the login can query the in-scope objects and catalog views.

Step 4 — Establish the credential path (Key Vault, safe by design)

1. Link an **Azure Key Vault** to the Purview account (or confirm one is linked).
2. Store the `PURVIEW_SCAN` password as a Key Vault **secret**, then create a Purview credential **by name** — `DGORACLE-Cred-Oracle-KV` (Basic authentication: the username inline, the password referenced from the Key Vault secret). **No password is ever placed in a script or document.**

Step 5 — Establish the SHIR, network path, and driver

1. Install and register a **self-hosted integration runtime (SHIR)** — `DGORACLE-SHIR` — on a machine

with **network line of sight** to the Oracle listener; confirm it shows **Running/Available** in the portal.

1. Open the firewall so **only** the SHIR host can reach the Oracle **listener port** (default `1521`; verify your instance's port).
1. Install the **Oracle JDBC driver** on the SHIR machine where Microsoft's current docs specify (**verify whether your SHIR version bundles it or requires a manual install**). A managed VNet runtime may be used instead for a cloud-reachable instance — verify.

Step 6 — Record the pilot scope

1. Run `DGORACLE-Set-Scope.ps1` to record the pilot Oracle source (host/service, port, the in-scope schemas, owner/steward, the credential name `DGORACLE-Cred-Oracle-KV`, and the scan-rule-set intent) as a scope manifest to configure against (read-only; produces the manifest you register against).

Step 7 — Register the pilot Oracle source into the Data Map

1. Register the pilot source into `DGORACLE-Collection-Pilot`. You may use the portal (**Register** → **Oracle**), or run `DGORACLE-Register-OracleSource.ps1` which **defaults to** `-WhatIf` and prints exactly what it would register (kind `Oracle`, the host/service, port, the collection, and the credential name); re-run with `-Commit` to register.
1. Run **Test connection** for the registered source and confirm it passes (the SHIR reaches the listener, the JDBC driver is present, and the Key Vault credential authenticates) before scanning. Do not scan until Test connection passes.

Step 8 — Configure and pilot the scan (bounded scope, pilot-first)

1. Define the scan configuration: a **scan rule set** named `DGORACLE-ScanRuleSet-Pilot` (start from the system default rule set), the **classifications** to apply (system classifications plus any custom rule named `DGORACLE-Classification-Custom-<name>`), and a **scoped** scan that targets only the pilot **schemas** — not every schema on the instance.
1. Trigger the scan **once** on the bounded scope (not scheduled yet) via `DGORACLE-Register-OracleSource.ps1 -ConfigureScan` (`-WhatIf` by default; `-Commit` to apply) or the portal. Pass/fail: the scan completes with **status Succeeded** and returns assets with classifications.
1. **Rollback:** if a scan misbehaves, over-reaches, or loads the database, cancel the run in the portal and set the scan to disabled; a scan does not modify source data, but cancel to stop resource consumption. Re-scope to fewer schemas and re-run.

Step 9 — Review scanned assets and classifications

1. Run `DGORACLE-Report.ps1` to list the pilot Oracle assets discovered (schemas/tables/columns), the

classifications applied, and any assets returned **unclassified**.

1. With the data owner, confirm the classifications are correct. Where a real internal pattern was missed, define a **custom classification rule** (`DGORACLE-Classification-Custom-<name>`), validate it against sample data, add it to `DGORACLE-ScanRuleSet-Pilot`, and re-scan the pilot scope.

Step 10 — Assign ownership on the Oracle assets

1. In the catalog, assign the **data owner** and **steward** recorded in the scope manifest to the scanned database/schemas so every governed Oracle asset has a named owner — ownership is what makes classification and later access decisions accountable.

Step 11 — Validate against the pilot gate

1. Complete the Testing Guide. Run `DGORACLE-Test-Readiness.ps1` for a consolidated PASS/GAP gate check.

The pilot passes only when: the Oracle source is registered and **Test connection** passes; the scan **Succeeded** and returned classified assets; and every scanned pilot asset has an owner assigned.

1. Remediate any gaps and re-run the relevant validation before proceeding. Do **not** widen scope until the gate passes.

Step 12 — Expand across the Oracle estate (enforce)

1. After the pilot passes, register further Oracle databases/schemas one grouping at a time, re-running `DGORACLE-Set-Scope.ps1`, `DGORACLE-Register-OracleSource.ps1`, and `DGORACLE-Report.ps1` for each new scope. Reuse the same Key Vault credential pattern (a per-database read login where the estate spans instances) and the same SHIR where it has line of sight.

1. Move scans from one-time to **scheduled** only once a scope is validated, so classifications stay current as Oracle data changes — and watch SHIR/database load as scope grows.

1. Keep ownership assigned on every newly scanned asset and track coverage with `DGORACLE-Report.ps1`.

Order of operations

```
Assess -> Design (collection/roles) -> Oracle read login -> Credential (Key Vault) -> SHIR + network + JDBC driver -> Record scope -> Register Oracle source -> Test connection -> Pilot scan (bounded schemas) -> Review classifications -> Assign ownership -> Validate gate -> Expand (scheduled scans)
```

5. Testing Guide — Oracle Data Governance: Microsoft Purview

Run these tests after the pilot scan (Implementation Steps 7–10) and again after each expansion scope. Each test confirms one part of the Oracle-governance workflow works end to end. All read-only checks are safe to repeat.

Test 1 — Account and roles are ready

Do: Run `DGORACLE-Discovery.ps1`. Expected: The Purview account endpoint is reachable, `DGORACLE-Collection-Pilot` exists, and least-privilege roles (Collection admin, Data source admin, Data curator, Data reader) are reported as assigned; **Data source admin** (needed to register/scan) is present. No required role is missing.

Test 2 — The Oracle connector is available

Do: In the Purview portal, open **Data Map** → **Sources** → **Register**. Expected: **Oracle** appears as a registerable source type for your tenant, and your Oracle database version is within the connector's currently supported range.

Test 3 — A SHIR is registered and running

Do: Run `DGORACLE-Discovery.ps1` and check the integration-runtime section; confirm in the portal. Expected: The self-hosted integration runtime `DGORACLE-SHIR` is registered and shows **Running/Available**, on a host with network line of sight to the Oracle listener, with the Oracle JDBC driver present.

Test 4 — The credential path works (Key Vault + Test connection)

Do: On the registered Oracle source, run **Test connection**; confirm the credential `DGORACLE-Cred-Oracle-KV` references the Key Vault secret by name. Expected: Purview authenticates to Oracle with the **named Key Vault credential** (Basic auth) over the SHIR network path and **Test connection passes**. No password is stored in any script or document.

Test 5 — The scan succeeds and returns assets

Do: Review the pilot scan run in the portal and run `DGORACLE-Report.ps1`. Expected: The scan status is **Succeeded**, the run scanned the intended bounded scope (the pilot schemas, not the whole instance), and assets (schemas/tables/columns) were returned into the Data Map.

Test 6 — Oracle assets are classified

Do: Run `DGORACLE-Report.ps1` and review the classification coverage for the pilot Oracle assets. Expected: System (and any custom) classifications are applied to the columns/tables that contain matching data; the count of unexpectedly **unclassified** assets is understood and accepted by the data owner.

Test 7 — Scanned Oracle assets have owners

Do: Open the pilot database/schemas in the catalog and run `DGORACLE-Report.ps1`. Expected: Every scanned pilot asset has a named **owner/steward** assigned (from the scope manifest); no governed Oracle

asset is unowned.

Test 8 — Consolidated readiness gate

Do: Run `DGORACLE-Test-Readiness.ps1 -PurviewAccount <account> -Collection DGORACLE-Collection-Pilot`. Expected: A single PASS/GAP summary confirming account reachable, roles present, a SHIR registered, the Oracle source registered and Test-connection-ready, at least one scan Succeeded, classification coverage > 0, and owners assigned — the pilot gate in one view.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–7 pass: connector available, SHIR running with driver, Key Vault credential Test connection passes, scan Succeeded with classified assets, and every scanned asset has an owner (Test 8 shows all PASS)
Expand pass	Tests 1–7 pass for each new Oracle scope, and scheduled scans are enabled only after the scope validates (with expected SHIR/database load accepted)

6. Troubleshooting Guide — Oracle Data Governance (Microsoft Purview)

Symptom → likely root cause → fix. Use `DGORACLE-Discovery.ps1` (account/roles/SHIR/sources), `DGORACLE-Test-Readiness.ps1` (consolidated PASS/GAP gate), and `DGORACLE-Report.ps1` (assets / classifications / owners) to confirm before and after each fix. The Oracle connector, supported versions, driver requirement, and authentication methods **evolve** — where a detail is flagged, **verify against current Microsoft documentation / your tenant** before changing the estate.

A. Account, collections & roles

`DGORACLE-Discovery.ps1` *can't reach the Purview account*

- **Endpoint / auth wrong. Fix:** confirm the account name/endpoint and that `Connect-MgGraph`/the data-plane sign-in succeeded with an account that can read the Data Map; re-run Discovery.

"You don't have permission" when registering or scanning

- **Missing Data source admin.** Registration/scan requires **Data source admin** on the collection.

Fix: `DGORACLE-Discovery.ps1` reports role coverage; assign Data source admin on `DGORACLE-Collection-Pilot` at the narrowest scope (Req/Step 2), not account-wide.

B. Connector availability

Oracle isn't offered under Data Map → Sources → Register

- **Connector not available for the tenant / region, or version unsupported. Fix:** **verify the

Oracle connector availability and supported Oracle versions for your tenant** in current Microsoft docs; confirm your database version is within the supported range before proceeding.

C. SHIR / network / driver

The self-hosted integration runtime shows offline or unavailable

- **Service stopped / registration lapsed / no outbound. Fix:** confirm the `DGORACLE-SHIR` service

is running on the host and re-check the portal; ensure the host has the required outbound connectivity to the Purview/managed service. Re-run `DGORACLE-Discovery.ps1` to confirm Running/Available.

Test connection fails with a network/timeout error

- **No line of sight to the listener / firewall. Fix:** confirm the SHIR host can reach the Oracle

listener port (default `1521` — verify your instance's port); open the firewall so **only** the SHIR host reaches the listener (Step 5).

Test connection fails with a driver/class-not-found error

- **Oracle JDBC driver missing on the SHIR machine. Fix:** install the **Oracle JDBC driver** where

Microsoft's current docs specify; **verify whether your SHIR version bundles it or requires a manual install**. Restart the SHIR after installing and retry Test connection.

D. Credential path

Test connection fails with an authentication error

- **Wrong login, insufficient grants, or Key Vault reference broken. Fix:** confirm `PURVIEW_SCAN`

has `CREATE SESSION` + `SELECT` on the in-scope objects and the data-dictionary views (Step 3; verify the exact grant list in current docs — **no DBA/SYS**); confirm the credential `DGORACLE-Cred-Oracle-KV` references the correct **Key Vault secret by name** and that Purview's managed identity can read that secret. **No password belongs in any script or document.**

E. Scan results

The scan status is Failed

- **Credential / driver / network (see C, D), or scope invalid. Fix:** open the failed run in the

portal for the error; resolve the connectivity/credential cause; confirm the scoped schemas exist and are readable by `PURVIEW_SCAN`; re-scan the bounded pilot scope.

The scan Succeeded but returned few/no assets

- **Scope too narrow or login can't see the objects. Fix:** confirm the scan targeted the intended

pilot schemas (not an empty schema); confirm `PURVIEW_SCAN` has `SELECT` on those objects; review with `DGORACLE-Report.ps1` and re-scope.

The scan over-reached / loaded the database

• **Scanned the whole instance or scheduled too aggressively. Fix:** cancel the run in the portal and

set the scan disabled (a scan does not modify source data — cancel only to stop resource use); re-scope to fewer schemas via `DGORACLE-Set-Scope.ps1`; keep scans **one-time** until the scope validates, then schedule (Step 12) and watch SHIR/database load.

F. Classification & ownership

Expected columns come back unclassified

• **Detector didn't match / custom pattern needed. Fix:** confirm the classification is in

`DGORACLE-ScanRuleSet-Pilot`; for a real internal pattern that was missed, define a **custom classification rule** (`DGORACLE-Classification-Custom-<name>`), validate against sample data, add it to the rule set, and re-scan (Step 9). The count of accepted-unclassified assets is a **data-owner determination**, not a technical one.

Scanned assets have no owner

• **Ownership not assigned after scan. Fix:** assign the **owner/steward** from the scope manifest to

the scanned database/schemas (Step 10); `DGORACLE-Report.ps1` lists assets missing an owner. No governed Oracle asset should be unowned.

Escalation checklist

1. `DGORACLE-Discovery.ps1` — account reachable, `DGORACLE-Collection-Pilot` present, Data source admin

assigned, a SHIR registered?

1. `DGORACLE-Test-Readiness.ps1 -PurviewAccount <account> -Collection DGORACLE-Collection-Pilot` — which

gate item is GAP (connector / SHIR / credential / scan / classification / owners)?

1. Portal Test connection — network (C), driver (C), or credential (D)?
2. Portal scan run + `DGORACLE-Report.ps1` — Succeeded, scoped, classified, owned?
3. Still stuck → **verify current Microsoft Oracle-connector guidance** (supported versions, driver,

grant list, auth methods) for your tenant before a broad change. Classification acceptance and ownership are **data-owner determinations** — this guide restores the *technical scan path*.

7. Best Practices — Oracle Data Governance: Microsoft Purview

• Confirm connector availability and version support before you design anything. The Oracle connector,

the supported Oracle versions, the JDBC-driver requirement, and the supported authentication methods are evolving — read them from the Purview registration UI and Microsoft's current docs for your tenant rather than assuming a value that may have moved.

- Keep the Oracle secret in Key Vault, referenced by name. Link an Azure Key Vault to the account, store

the scanning login's password there, and reference it by name (`DGORACLE-Cred-Oracle-KV`) — this is both the secure pattern and what keeps the provided scripts secret-free. Never put an Oracle password in a script, a scan definition, or a document.

- Use a least-privilege, read-only scanning login — never a DBA account. Grant `CREATE SESSION` plus

`SELECT` on just the in-scope schemas/objects and the data-dictionary views the scanner reads; verify the exact grant list in Microsoft's current docs. Widen access deliberately as you expand, one scope at a time.

- Right-size the network path and lock the firewall down. Stand up a self-hosted integration runtime

(SHIR) with line of sight to the Oracle listener, and open the listener port (default 1521) **only** from the SHIR host — no broader. Confirm the SHIR shows Running/Available before you scan.

- Get the JDBC driver right on the SHIR. Missing or mismatched Oracle JDBC driver on the SHIR machine is

the most common cause of a failed Test connection; verify whether your SHIR version bundles the driver or requires a manual install, and place it exactly where Microsoft's current docs specify.

- Pilot every scan on a bounded scope. Scoped, one-time scans on a few schemas prove classification,

connectivity, driver, and database load before you point a scan at an entire instance; move to scheduled scans only after a scope validates.

- Mind database and SHIR load. Scanning reads live Oracle objects and sample data through the SHIR;

agree the scan window with the DBA, keep scans one-time until validated, and watch instance load as scope grows.

- Trust system classifications first, add custom sparingly. The built-in classifications cover most

regulated data; define a custom classification rule (`DGORACLE-Classification-Custom-<name>`) only for a real internal pattern (an internal account-number or customer-ID format), and validate it against sample data before Oracle-wide use.

- Every scanned Oracle asset gets an owner and a steward. Ownership is what makes classification and

access decisions accountable; an unowned database or schema in the catalog is a governance gap, not a governed asset.

- Treat determinations as someone else's job. The solution registers, scans, classifies, and catalogs

Oracle data; the business/records/legal owners decide sensitivity, retention, and access. Keep that separation explicit so technical scanning is never mistaken for a compliance ruling.

- Verify licensing, pricing, and portal paths as you go. The connector and the consumption billing are

evolving — confirm current availability and pricing in Microsoft's docs and the portal rather than assuming a path, a supported version, or a price is permanent.

8. Deliverables Summary — Oracle Data Governance: Microsoft Purview

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	Registering, scanning, and classifying Oracle databases into the Purview Data Map — the connector, the safe Key Vault credential + SHIR model, and how it complements M365 Purview (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: design, read login, Key Vault credential, SHIR + network + driver, register, pilot-scan, classify, assign ownership, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Named artifacts

Artifact	Name
Pilot collection	`DGORACLE-Collection-Pilot`
Oracle credential (references the Key Vault secret by name)	`DGORACLE-Cred-Oracle-KV`
Self-hosted integration runtime	`DGORACLE-SHIR`
Scan rule set	`DGORACLE-ScanRuleSet-Pilot`
Custom classification rule (as needed)	`DGORACLE-Classification-Custom-<name>`
Example read-only Oracle login (created in Oracle)	`PURVIEW_SCAN`

PowerShell scripts

Script	What it does
`DGORACLE-Discovery.ps1`	Reports readiness: account endpoint reachable, collections + role coverage, registered integration runtimes (SHIR), and registered Oracle source counts (Purview data-plane REST via an Azure token; read-only)
`DGORACLE-Set-Scope.ps1`	Records the pilot/expansion Oracle scope — the host/service, port, in-scope schemas, owners, collection, credential name, and scan-rule-set intent — as a scope manifest to configure against (read-only)
`DGORACLE-Register-OracleSource.ps1`	Registers an Oracle source and configures a scoped scan; **defaults to** `-WhatIf` (prints the plan), applies only with `-Commit`; credential referenced by name — no secrets in code
`DGORACLE-Report.ps1`	Scan + classification coverage report for Oracle assets: scan run status, classified vs unclassified assets, and owner coverage (read-only)
`DGORACLE-Test-Readiness.ps1`	Consolidated PASS/GAP pilot-gate check across account, roles, SHIR, Oracle source registration/Test-connection readiness, scan status, classification coverage, and ownership (read-only)

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable method following the model: Assess → Design → Configure → Pilot → Validate → Expand.

Outcome

Oracle data governed from Purview — databases and schemas registered into the Data Map, scanned and classified, and owned — proven on a bounded pilot before you scale, with the Oracle blind spot closed and regulatory, records, and access determinations left to the owners who make them.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DGORACLE-Discovery.ps1

Purpose: Reports readiness so you can assess current state before registering an Oracle source or scanning: whether the Microsoft Purview account data-plane endpoint is reachable, which collections exist and their role-assignment coverage (especially Data source admin, needed to register/scan), which integration runtimes are registered (you need a self-hosted IR for Oracle), and how many Oracle sources are already registered. Read-only — it makes no change to the account, sources, or catalog. Supports the Assess step. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGORACLE-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the collections. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary: endpoint reachable (PASS/GAP), collection count and per-collection role coverage, registered integration runtimes (self-hosted IR present?), and a count of registered Oracle sources — with a PASS/GAP line per signal to record against the Requirements checklist. .NOTES Purview — Oracle Data Governance. Read-only against the Purview data-plane REST API. Verify your licensing/tenant: the Data Governance experience, its endpoints, and the Oracle connector are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGORACLE-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the collections. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary: endpoint reachable (PASS/GAP), collection count and per-collection role coverage, registered integration runtimes (self-hosted IR present?), and a count of registered Oracle sources — with a PASS/GAP line per signal to record against the Requirements checklist. .NOTES Purview — Oracle Data Governance. Read-only against the Purview data-plane REST API. Verify your licensing/tenant: the Data Governance experience, its endpoints, and the Oracle connector are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGORACLE-Register-OracleSource.ps1

Purpose: Registers an Oracle data source into a Data Map collection and (with -ConfigureScan) prepares a scoped, one-time scan using a named scan rule set and self-hosted integration runtime. SAFE BY DEFAULT: with no -Commit it runs in -WhatIf mode and only prints the exact registration/scan payload it WOULD send — it changes nothing. Pass -Commit to apply. No Oracle secret is ever placed in this script: authentication uses a Purview credential (Basic auth) that references an Azure Key Vault secret BY NAME. HOW TO RUN Preview an Oracle registration (default, no change): powershell -ExecutionPolicy Bypass -File .\DGORACLE-Register-OracleSource.ps1 -PurviewAccount contoso-purview `-Collection DGORACLE-Collection-Pilot -SourceName oracle-oradb01` -OracleHost oradb01.contoso.example -Service ORCLPDB1 -Port 1521 ` -Credential DGORACLE-Cred-Oracle-KV

-IntegrationRuntime DGORACLE-SHIR Apply + configure a scoped scan: ... same args ... -ConfigureScan -ScanRuleSet DGORACLE-ScanRuleSet-Pilot -Schema HR,FINANCE -Commit Requires Az.Accounts and Connect-AzAccount with Data source admin on the collection. EXPECTED RESULTS Default: a printed "[WhatIf] would register / would configure scan" plan and NO change. With -Commit: the source is registered (HTTP 200/201) and, with -ConfigureScan, a one-time scan definition is created; the script prints the resulting object names. Run Test connection in the portal, then trigger/monitor the scan; review with DGORACLE-Report.ps1. .NOTES Purview — Oracle Data Governance. Writes gated behind -Commit; -WhatIf is the default. Verify the exact source "kind" (Oracle), the connectionProperty/scan field names, the credentialType, and the current scan API versions for your tenant in Microsoft's docs — the connector is evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$Collection, [Parameter(Mandatory = $true)][string]$SourceName, [Parameter(Mandatory = $true)][string]$OracleHost, # Oracle host, e.g. oradb01.contoso.example [Parameter(Mandatory = $true)][string]$Service, # service name or SID, e.g. ORCLPDB1 [int]$Port = 1521, [Parameter(Mandatory = $true)][string]$Credential, # NAME of the Purview credential (Key Vault secret, Basic auth) [string]$IntegrationRuntime = 'DGORACLE-SHIR', [switch]$ConfigureScan, [string]$ScanRuleSet = 'DGORACLE-ScanRuleSet-Pilot', [string[]]$Schema = @(), # in-scope schemas to bound the scan [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

How to run: Preview an Oracle registration (default, no change): powershell -ExecutionPolicy Bypass -File .\DGORACLE-Register-OracleSource.ps1 -PurviewAccount contoso-purview `-Collection DGORACLE-Collection-Pilot -SourceName oracle-oradb01` -OracleHost oradb01.contoso.example -Service ORCLPDB1 -Port 1521 ` -Credential DGORACLE-Cred-Oracle-KV -IntegrationRuntime DGORACLE-SHIR Apply + configure a scoped scan: ... same args ... -ConfigureScan -ScanRuleSet DGORACLE-ScanRuleSet-Pilot -Schema HR,FINANCE -Commit Requires Az.Accounts and Connect-AzAccount with Data source admin on the collection. EXPECTED RESULTS Default: a printed "[WhatIf] would register / would configure scan" plan and NO change. With -Commit: the source is registered (HTTP 200/201) and, with -ConfigureScan, a one-time scan definition is created; the script prints the resulting object names. Run Test connection in the portal, then trigger/monitor the scan; review with DGORACLE-Report.ps1. .NOTES Purview — Oracle Data Governance. Writes gated behind -Commit; -WhatIf is the default. Verify the exact source "kind" (Oracle), the connectionProperty/scan field names, the credentialType, and the current scan API versions for your tenant in Microsoft's docs — the connector is evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$Collection, [Parameter(Mandatory = $true)][string]$SourceName, [Parameter(Mandatory = $true)][string]$OracleHost, # Oracle host, e.g. oradb01.contoso.example [Parameter(Mandatory = $true)][string]$Service, # service name or SID, e.g. ORCLPDB1 [int]$Port = 1521, [Parameter(Mandatory = $true)][string]$Credential, # NAME of the Purview credential (Key Vault secret, Basic auth) [string]$IntegrationRuntime = 'DGORACLE-SHIR', [switch]$ConfigureScan, [string]$ScanRuleSet = 'DGORACLE-ScanRuleSet-Pilot', [string[]]$Schema = @(), # in-scope schemas to bound the scan [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

DGORACLE-Report.ps1

Purpose: Reports how the Oracle governance pilot is doing: for the Oracle source(s), the latest scan run status, the assets returned into the Data Map (schemas/tables/columns), how many carry a classification vs are unclassified, and how many carry an owner/steward. Read-only — it queries the Purview data-plane REST API and changes nothing. Use it after a pilot scan and after each expansion scope to confirm

classification and ownership coverage. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGORACLE-Report.ps1 -PurviewAccount contoso-purview Optional: -SourceName oracle-oradb01 to focus on one source. Requires Az.Accounts and Connect-AzAccount with at least Data reader on the collection. Acquires a data-plane token (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS Per Oracle source: latest scan run status (ideally Succeeded), asset count, classified vs unclassified counts, and owner-coverage count — a coverage snapshot to check against the pass/fail gate (scan Succeeded, classification coverage > 0, every asset owned). .NOTES Purview — Oracle Data Governance. Read-only. Verify the current scan/search API versions and the search filter fields for your tenant — the Data Governance search surface is evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$SourceName, # optional: focus on one Oracle source [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGORACLE-Report.ps1 -PurviewAccount contoso-purview Optional: -SourceName oracle-oradb01 to focus on one source. Requires Az.Accounts and Connect-AzAccount with at least Data reader on the collection. Acquires a data-plane token (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS Per Oracle source: latest scan run status (ideally Succeeded), asset count, classified vs unclassified counts, and owner-coverage count — a coverage snapshot to check against the pass/fail gate (scan Succeeded, classification coverage > 0, every asset owned). .NOTES Purview — Oracle Data Governance. Read-only. Verify the current scan/search API versions and the search filter fields for your tenant — the Data Governance search surface is evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$SourceName, # optional: focus on one Oracle source [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGORACLE-Set-Scope.ps1

Purpose: Captures and writes a scope manifest for the Oracle pilot (or an expansion wave): the target collection, the Oracle source (host, service/SID, listener port, version) and the schemas to scan (with owner/steward and the credential name it uses), and the scan-rule-set intent. Read-only against the tenant — it writes only a local JSON manifest you register and scan against; it does not change the Purview account or the Oracle database, and it holds NO secret (the credential is referenced by name). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGORACLE-Set-Scope.ps1 -Collection DGORACLE-Collection-Pilot `-Host oradb01.contoso.example -Service ORCLPDB1 -Port 1521` -Schema @('HR','FINANCE') -Credential DGORACLE-Cred-Oracle-KV ` -IntegrationRuntime DGORACLE-SHIR -OutFile .\oracle-pilot-scope.json EXPECTED RESULTS An oracle-pilot-scope.json manifest listing the collection, the Oracle source (host/service/port), the in-scope schemas, owner/steward placeholders, the credential name, the SHIR, and the scan rule set, plus a printed summary — used as the input to DGORACLE-Register-OracleSource.ps1. .NOTES Purview — Oracle Data Governance. Local manifest only; no tenant or Oracle change; no secrets. Names are white-label placeholders (contoso.example). Assign a real owner/steward before registering. > [CmdletBinding()] param([string]$Collection = 'DGORACLE-Collection-Pilot', [Parameter(Mandatory = $true)][string]$Host, # Oracle host, e.g. oradb01.contoso.example [Parameter(Mandatory = $true)][string]$Service, # service name or SID, e.g. ORCLPDB1 [int]$Port = 1521, # listener port (verify your instance) [string[]]$Schema = @(), # in-scope schemas, e.g. 'HR','FINANCE' [string]$Credential = 'DGORACLE-Cred-Oracle-KV', # NAME of the Purview credential (Key Vault secret, Basic auth) [string]$IntegrationRuntime = 'DGORACLE-SHIR', [string]$ScanRuleSet = 'DGORACLE-ScanRuleSet-Pilot', [string]$OutFile = '.\oracle-pilot-scope.json')

How to run: powershell -ExecutionPolicy Bypass -File .\DGORACLE-Set-Scope.ps1 -Collection DGORACLE-Collection-Pilot `-Host oradb01.contoso.example -Service ORCLPDB1 -Port

1521 -Schema @('HR','FINANCE') -Credential DGORACLE-Cred-Oracle-KV ` -IntegrationRuntime DGORACLE-SHIR -OutFile .\oracle-pilot-scope.json EXPECTED RESULTS An oracle-pilot-scope.json manifest listing the collection, the Oracle source (host/service/port), the in-scope schemas, owner/steward placeholders, the credential name, the SHIR, and the scan rule set, plus a printed summary — used as the input to DGORACLE-Register-OracleSource.ps1. .NOTES Purview — Oracle Data Governance. Local manifest only; no tenant or Oracle change; no secrets. Names are white-label placeholders (contoso.example). Assign a real owner/steward before registering. > [CmdletBinding()] param([string]$Collection = 'DGORACLE-Collection-Pilot', [Parameter(Mandatory = $true)][string]$Host, # Oracle host, e.g. oradb01.contoso.example [Parameter(Mandatory = $true)][string]$Service, # service name or SID, e.g. ORCLPDB1 [int]$Port = 1521, # listener port (verify your instance) [string[]]$Schema = @(), # in-scope schemas, e.g. 'HR','FINANCE' [string]$Credential = 'DGORACLE-Cred-Oracle-KV', # NAME of the Purview credential (Key Vault secret, Basic auth) [string]$IntegrationRuntime = 'DGORACLE-SHIR', [string]$ScanRuleSet = 'DGORACLE-ScanRuleSet-Pilot', [string]$OutFile = '.\oracle-pilot-scope.json')

DGORACLE-Test-Readiness.ps1

Purpose: Runs the pilot gate in one view: is the Purview account reachable, are the least-privilege roles present on the pilot collection, is a self-hosted integration runtime registered, is the Oracle source registered (and does it have a scan that Succeeded), and is there classification + owner coverage on the scanned assets. Read-only — it only reads the Purview data-plane REST API and prints a PASS/GAP summary; it changes nothing and holds no secret. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGORACLE-Test-Readiness.ps1 -PurviewAccount contoso-purview ` -Collection DGORACLE-Collection-Pilot Requires Az.Accounts and Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A single PASS/GAP block: account reachable, Data source admin present on the collection, a SHIR registered, at least one Oracle source registered, at least one scan Succeeded, classification coverage > 0, and owners assigned — the pilot gate. Any GAP names what to remediate before you widen scope. .NOTES Purview — Oracle Data Governance. Read-only. Verify current API versions and search fields for your tenant. This is a readiness gate, not a compliance determination — sensitivity/retention/access determinations stay with the data owners, compliance, records, privacy, and legal functions. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection = 'DGORACLE-Collection-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGORACLE-Test-Readiness.ps1 -PurviewAccount contoso-purview ` -Collection DGORACLE-Collection-Pilot Requires Az.Accounts and Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A single PASS/GAP block: account reachable, Data source admin present on the collection, a SHIR registered, at least one Oracle source registered, at least one scan Succeeded, classification coverage > 0, and owners assigned — the pilot gate. Any GAP names what to remediate before you widen scope. .NOTES Purview — Oracle Data Governance. Read-only. Verify current API versions and search fields for your tenant. This is a readiness gate, not a compliance determination — sensitivity/retention/access determinations stay with the data owners, compliance, records, privacy, and legal functions. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection = 'DGORACLE-Collection-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")